

CMP_SC/ECE 4080/7080 – A3

Speeding Things Up with GPUs

Spring 2026

Overview

Course context

In this assignment, you will compare multicore CPU threading and GPU acceleration on a simple but important HPC workload: a **2D stencil computation**. You will work in **C** or **C++** and implement:

- a threaded CPU version
- a GPU version using **either CUDA or OpenACC**

A correct **serial CPU reference implementation will be provided to you** as the starting point.

Learning goals

By the end of this assignment, you should be able to:

- understand a regular grid-based HPC computation
- parallelize a serial implementation for a multicore CPU
- offload the computation to a GPU with CUDA or OpenACC
- measure performance carefully
- explain why a GPU can outperform CPU threading on large problems

The computation

You are given two 2D arrays, A and B, of size $n_x \times n_y$, stored as single-precision floats in a flat 1D row-major layout.

For each time step, every interior cell is updated as:

$$B[i][j] = 0.2f * (A[i][j] + A[i-1][j] + A[i+1][j] + A[i][j-1] + A[i][j+1]);$$

for $1 \leq i < n_x - 1$ and $1 \leq j < n_y - 1$.

Boundary cells remain fixed. After each time step, the roles of A and B are swapped.

You can think of this as either:

- smoothing a large grayscale image, or
- simulating heat diffusion on a plate

The math is simple, but the performance behavior is very interesting.

What is provided

You will be given a correct serial CPU reference implementation, for example:

- stencil_serial.cpp
- timer.h or equivalent timing utility
- a simple Makefile or compile command example
- a short README describing the input format and expected output

The provided serial program will:

- allocate and initialize the arrays
- run the time-stepping loop
- compute a checksum
- print timing information

You should use this implementation as:

- the correctness reference
- the baseline for performance comparison
- the starting point for your threaded and GPU versions

You may study the serial code and reuse non-parallel helper code such as initialization, memory allocation, argument parsing, and checksum calculation.

What you must implement

You must write **two new implementations** based on the provided serial reference.

1. Threaded CPU version

Write a threaded CPU version using **OpenMP**.

This version should preserve the same numerical behavior as the serial reference and produce a matching checksum within floating-point tolerance.

2. GPU version

Write a GPU version using **either**:

- **CUDA**, or
- **OpenACC**

This version should also match the serial reference within floating-point tolerance.

Hint

For the GPU version, be careful where the arrays are when doing the computation. If you can keep things on the GPU with no back and forth, it will improve performance.

Sample Output – Starter Code

```
[jer676@g003 hpc_stencil_single_one_loop]$ ./run_benchmarks.sh
Writing benchmark output to results/bench_20260420_153227.txt
===== ./stencil_serial 8192 8192 1000 =====
program=stencil_serial
nx=8192
ny=8192
steps=1000
runtime_seconds=37.316254
checksum=419021946.6262273192

===== ./stencil_omp 8192 8192 1000 16 =====
program=stencil_omp
nx=8192
ny=8192
steps=1000
threads=16
runtime_seconds=37.299151
checksum=419021946.6262273192

===== ./stencil_acc 8192 8192 1000 =====
program=stencil_acc
nx=8192
ny=8192
steps=1000
compute_seconds=37.822447
end_to_end_seconds=37.822447
checksum=419021946.6262273192
```

Sample Output - Solution

```
[jer676@g003 hpc_stencil_solution_omp_acc]$ ./run_benchmarks.sh
Writing benchmark output to results/bench_20260420_152208.txt
===== ./stencil_serial 8192 8192 1000 =====
program=stencil_serial
nx=8192
ny=8192
steps=1000
runtime_seconds=44.013062
checksum=419021946.6262273192

===== ./stencil_omp 8192 8192 1000 16 =====
program=stencil_omp
nx=8192
ny=8192
steps=1000
threads=16
runtime_seconds=7.367198
checksum=419021946.6262273192

===== ./stencil_acc 8192 8192 1000 =====
program=stencil_acc
nx=8192
ny=8192
steps=1000
compute_seconds=2.925308
end_to_end_seconds=3.291347
checksum=419021946.6263915300
```

How to test

You should run your tests on Hellbender using these SLURM parameters:

```
srun --partition=rss-class --gres=gpu --cpus-per-task=16
```

If you want to just get a bash prompt and run everything interactively, that is fine:

```
srun --partition=rss-class --gres=gpu --cpus-per-task=16 \  
--interactive --preserve-env --pty /bin/bash
```

Don't forget that to build, you will need one (or both) of:

```
module load nvhpc  
module load cuda
```

You can use the provided run_benchmarks.sh to provide the requested parameters.

What to submit

Commit and push everything to osgit. You should include a text file or PDF in your submission that shows a bench mark run from run_benchmarks.sh. At the bottom of that PDF or text file, add a paragraph explaining why you things improve (or not) from serial to cpu-threaded (OpenMP) and from cpu-threaded to gpu accelerated.